Министерство образования Республики Беларусь

Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра Информатики
Дисциплина «Программирование мобильных приложений»

ОТЧЁТ
к лабораторной работе №1
на тему
«КОНВЕРТЕР ЕДИНИЦ ИЗМЕРЕНИЯ/ВАЛЮТ»
БГУИР 6-05-0612-02 23

Выполнил студент группы 353503
СЕБЕЛЕВ Дмитрий Юрьевич

_____
(дата, подпись студента)

Проверил ассистент каф. информатики
БУРАКОВ Дмитрий Игоревич

_____
(дата, подпись преподавателя)

Минск 2026

СОДЕРЖАНИЕ

1 Постановка задачи 3
2 Выполнение работы 4
2.1 Архитектура и состав приложения 4
2.2 Отличия бесплатной и премиальной версий 4
2.3 Диаграмма объектов 6
Вывод 8
Приложение А (обязательное) Листинг программного кода 9

1 ПОСТАНОВКА ЗАДАЧИ

Целью лабораторной работы являлась разработка мобильного приложения – конвертера единиц измерения и валют – для операционной системы *Android*. Приложение должно поддерживать перевод между единицами расстояния, массы и денежными единицами, обеспечивать корректную обработку пользовательского ввода, а также предоставлять расширенный набор функций в рамках платной версии.

В ходе выполнения работы требовалось спроектировать архитектуру приложения с применением компонентной модели фрагментов и разделяемой модели представления, реализовать логику конвертации для нескольких категорий величин, разработать два варианта сборки – бесплатный и премиальный – и подготовить диаграмму объектов, отражающую состояние системы в момент выполнения операции конвертации.

2 ВЫПОЛНЕНИЕ РАБОТЫ

2.1 Архитектура и состав приложения

Разработанное приложение построено на основе фрагментной архитектуры платформы *Android*. Единственная активность является точкой входа и контейнером для двух фрагментов. Один из них отвечает за отображение данных, а второй реализует экранную клавиатуру. Взаимодействие между этими компонентами организовано через общую модель представления, привязанную к жизненному циклу активности, что исключает дублирование состояния и обеспечивает реактивное обновление интерфейса при изменении входных данных.

Логика конвертации вынесена в отдельные классы – по одному на каждую поддерживаемую категорию. В приложении реализованы преобразования для расстояния, массы и валют. Для каждой категории используется собственный конвертер. Он получает числовое значение и две единицы измерения, после чего возвращает объект результата, содержащий исходное и преобразованное значения вместе со ссылками на соответствующие единицы. Единицы измерения описываются моделью, хранящей идентификатор, отображаемое наименование, символ и идентификатор категории. Категории конвертации описываются отдельной моделью и служат для группировки единиц в пользовательском интерфейсе.

Фрагмент клавиатуры передаёт нажатые символы в модель представления, которая формирует строку ввода и инициирует пересчёт. Фрагмент данных наблюдает за состоянием модели представления и отображает актуальные значения – выбранную категорию, исходную и результирующую единицы. Поля ввода и вывода работают совместно с собственной экранной клавиатурой приложения, что отделяет пользовательский ввод от основной области отображения данных и делает интерфейс более целостным. Такое разделение обязанностей упрощает сопровождение кода и позволяет независимо изменять представление и бизнес-логику.

2.2 Отличия бесплатной и премиальной версий

Приложение выпускается в двух вариантах сборки. На рисунке 2.1 представлен интерфейс бесплатной версии, а на рисунке 2.2 – интерфейс премиальной.

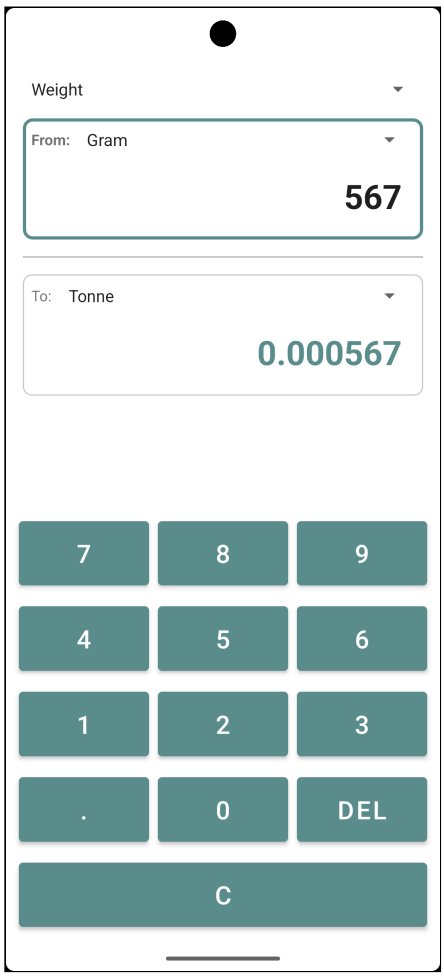

Рисунок 2.1 – Интерфейс *free* версии приложения

Бесплатная версия предоставляет базовый набор функций. В ней реализованы ввод значения через экранную клавиатуру, выбор категории и единиц измерения, а также отображение результата конвертации. Такой вариант сборки позволяет использовать приложение как универсальный конвертер без дополнительных сервисных возможностей.

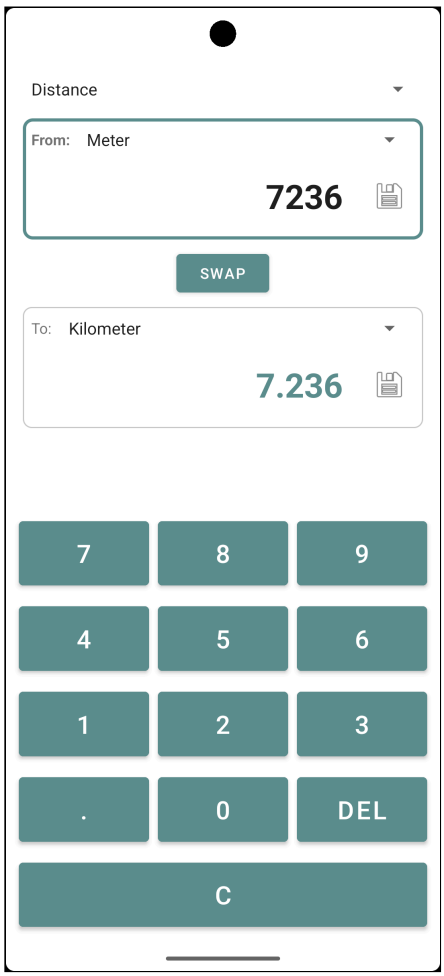

Рисунок 2.2 – Интерфейс *premium* версии приложения

Премиальная версия дополнена возможностью быстрой замены местами исходной и целевой единиц измерения, а также функциями копирования входного значения и результата в буфер обмена. Технически различие между версиями организовано через механизм product flavors и отдельные наборы исходных файлов и ресурсов. Благодаря этому базовая и расширенная функциональность поддерживаются в рамках одного проекта без дублирования общей логики приложения. Таким образом, премиальная версия ориентирована на пользователей, которым требуется более высокая скорость работы и удобство при частом использовании приложения.

2.3 Диаграмма объектов

На рисунке 2.3 представлена диаграмма объектов приложения, отражающая состояние системы в момент выполнения конкретной операции конвертации – перевода одного километра в метры. Диаграмма показывает, как связаны между собой активные экземпляры основных компонентов. На ней представлены фрагменты пользовательского интерфейса, модель представления, конвертер расстояния, объект результата и два объекта единиц измерения.

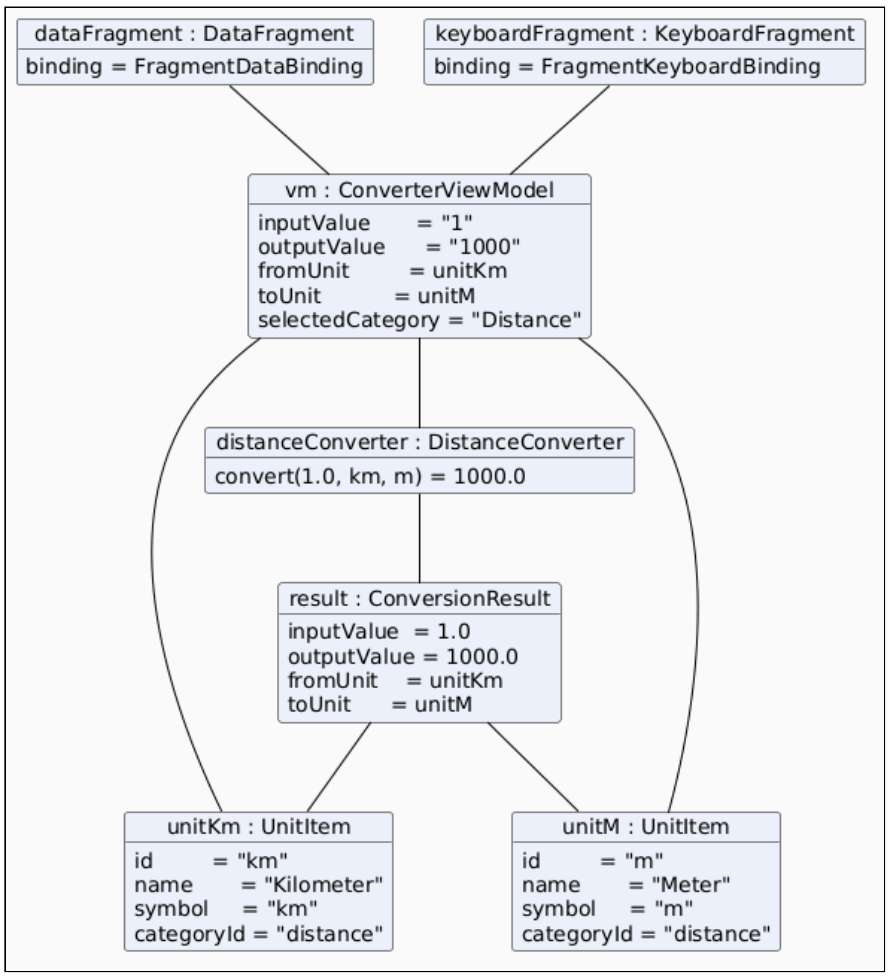

Рисунок 2.3 – Диаграмма объектов приложения

Диаграмма иллюстрирует, что оба фрагмента взаимодействуют с единственным экземпляром модели представления. Фрагмент клавиатуры передаёт нажатия клавиш, а фрагмент данных считывает результат. Модель представления делегирует вычисление конвертеру, который создаёт объект результата, связывающий входное и выходное значения с соответствующими единицами измерения.

ВЫВОД

В ходе выполнения лабораторной работы было разработано мобильное приложение – конвертер единиц измерения и валют для платформы *Android*. Реализована фрагментная архитектура с разделяемой моделью представления, обеспечивающая чёткое разграничение между логикой конвертации и уровнем отображения. Конвертация поддерживается для трёх категорий. В приложении реализованы преобразования расстояния, массы и валют. Каждая категория реализована в виде отдельного класса, принимающего на вход числовое значение и выбранные единицы и возвращающего структурированный объект результата.

Приложение выпущено в двух вариантах сборки – бесплатном и премиальном. Премиальная версия расширяет базовую функциональность возможностью перестановки единиц и копирования данных в буфер обмена, что повышает удобство использования в сценариях с частыми пересчётами. Подготовленная диаграмма объектов наглядно демонстрирует взаимодействие компонентов системы в момент выполнения операции и подтверждает корректность спроектированной архитектуры.

ПРИЛОЖЕНИЕ А
(обязательное)
Листинг программного кода

Листинг А.1 – Код метода *swapUnits*

```
  fun swapUnits() {
      val tempFrom = _fromUnit.value
      val tempTo = _toUnit.value

      val conv = _conversionResult.value
      val newInputValue: BigDecimal? = conv?.outputValue ?:
conv?.inputValue

      _fromUnit.value = tempTo
      _toUnit.value = tempFrom

      if (newInputValue == null) {
          _inputValue.value = ""
          _outputValue.value = ""
          _conversionResult.value = null
          return
      }

      val category = _selectedCategory.value
      val converter = category?.let { converters[it.id] }
      if (converter == null) {
          _error.value = "Converter not found"
          return
      }

      _inputValue.value = formatNumberForEditing(newInputValue)

      try {
          val result = converter.convert(newInputValue,
_fromUnit.value!!, _toUnit.value!!)
          _outputValue.value = formatNumberForEditing(result)

          _conversionResult.value = ConversionResult(
              inputValue = newInputValue,
              outputValue = result,
              fromUnit = _fromUnit.value!!,
              toUnit = _toUnit.value!!,
              formattedResult = "${formatNumber(result)}
${_toUnit.value!!.symbol}"
          )
          _error.value = null
      } catch (e: Exception) {
          _error.value = "Conversion error: ${e.message}"
      }
  }
```

Листинг А.2 – Код метода *performConvertion*

```
  private fun performConversion() {
      val category = _selectedCategory.value ?: return
      val from = _fromUnit.value ?: return
      val to = _toUnit.value ?: return
      val input = _inputValue.value ?: ""

      if (input.isEmpty() || input == ".") {
          _conversionResult.value = null
          _outputValue.value = ""
          return
      }

      if (input.length > 500) {
          _error.value = "Input too long"
          return
      }

      val inputBigDecimal = try { input.toBigDecimal() } catch (e:
Exception) { null }
      if (inputBigDecimal == null) {
          _error.value = "Invalid input"
          _conversionResult.value = null
          _outputValue.value = ""
          return
      }

      val converter = converters[category.id]
      if (converter == null) {
          _error.value = "Converter not found"
          _conversionResult.value = null
          return
      }

      try {
          val result = converter.convert(inputBigDecimal, from, to)
          val formatted = formatResult(result, to)

          _conversionResult.value = ConversionResult(
              inputValue = inputBigDecimal,
              outputValue = result,
              fromUnit = from,
              toUnit = to,
              formattedResult = formatted
          )
          _outputValue.value = formatNumber(result)
          _error.value = null
      } catch (e: Exception) {
          _error.value = "Conversion error: ${e.message}"
          _conversionResult.value = null
      }
  }
```

Листинг А.3 – Код конвертера дистанции

```
class DistanceConverter : Converter {
```

```
    companion object {
        const val CATEGORY_ID = "distance"

        // Conversion factors to meters (base unit)
        private val TO_METERS = mapOf(
            "m" to BigDecimal("1.0"),
            "km" to BigDecimal("1000.0"),
            "cm" to BigDecimal("0.01"),
            "mm" to BigDecimal("0.001"),
            "mi" to BigDecimal("1609.344"),
            "yd" to BigDecimal("0.9144"),
            "ft" to BigDecimal("0.3048"),
            "in" to BigDecimal("0.0254")
        )
    }

    private val units = listOf(
        UnitItem("m", "Meter", "m", CATEGORY_ID),
        UnitItem("km", "Kilometer", "km", CATEGORY_ID),
        UnitItem("cm", "Centimeter", "cm", CATEGORY_ID),
        UnitItem("mm", "Millimeter", "mm", CATEGORY_ID),
        UnitItem("mi", "Mile", "mi", CATEGORY_ID),
        UnitItem("yd", "Yard", "yd", CATEGORY_ID),
        UnitItem("ft", "Foot", "ft", CATEGORY_ID),
        UnitItem("in", "Inch", "in", CATEGORY_ID)
    )

    override fun getCategoryId(): String = CATEGORY_ID

    override fun getAvailableUnits(): List<UnitItem> = units

    override fun convert(value: BigDecimal, fromUnit: UnitItem,
toUnit: UnitItem): BigDecimal {
        val fromFactor = TO_METERS[fromUnit.id] ?: BigDecimal.ONE
        val toFactor = TO_METERS[toUnit.id] ?: BigDecimal.ONE

        // Convert to meters first, then to target unit
        val valueInMeters = value.multiply(fromFactor)
        return valueInMeters.divide(toFactor, 30,
RoundingMode.HALF_UP)
    }
}
```